Assignment-14.5

Akshay Patel

2303A51745

Batch-11

Task 1 – Image Gallery Website Create an image gallery web page.

Prompt:

<!--

Create an image gallery web page using HTML and CSS.

The gallery should display images in a grid layout using CSS Grid.

Add hover zoom or overlay effects.

Ensure mobile responsiveness.

Don't add image URLs in the code; leave the image src empty so I can add later.

—>

Code:

```html
<!-- Prompt:
Create a responsive image gallery using CSS Grid.
Add smooth hover zoom and caption overlay.
Use modern styling and spacing.
-->

<!DOCTYPE html>
<html>
<head>
<title>Modern Gallery</title>
<style>
body {
  margin: 0;
  font-family: 'Segoe UI';
  background: #111;
  color: white;
}

.gallery {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(220px, 1fr));
  gap: 20px;
  padding: 30px;
}

.card {
  position: relative;
  border-radius: 10px;
  overflow: hidden;
}

.card img {
  width: 100%;
  height: 200px;
  object-fit: cover;
  transition: 0.4s;
}

.card:hover img {
  transform: scale(1.2) rotate(2deg);
}

.caption {
  position: absolute;
  bottom: 0;
  width: 100%;
  background: linear-gradient(transparent, black);
  padding: 10px;
  opacity: 0;
  transition: 0.3s;
}
```

```
.card:hover .caption {
  opacity: 1;
}
</style>
</head>

<body>

<h2 style="text-align:center;">Creative Gallery</h2>

<div class="gallery">
  <div class="card">
    <img src="img1.jpg">
    <div class="caption">Nature</div>
  </div>
</div>

</body>
</html>
```

Output:

Explanation:

Creates a responsive image gallery using CSS Grid to arrange images neatly.
Adds hover effects like zoom and overlay to improve visual interaction.

Task 2 – Profile Card Generator

Create a profile card web page.

Prompt:

<!--
Create a profile card web page.
Display user image, name
role, and social media links.
Suggest modern UI colors and fonts.
Design the card using Flexbox.
Add hover animation and shadow effects.
Give me clean and responsive HTML + CSS code.
—>

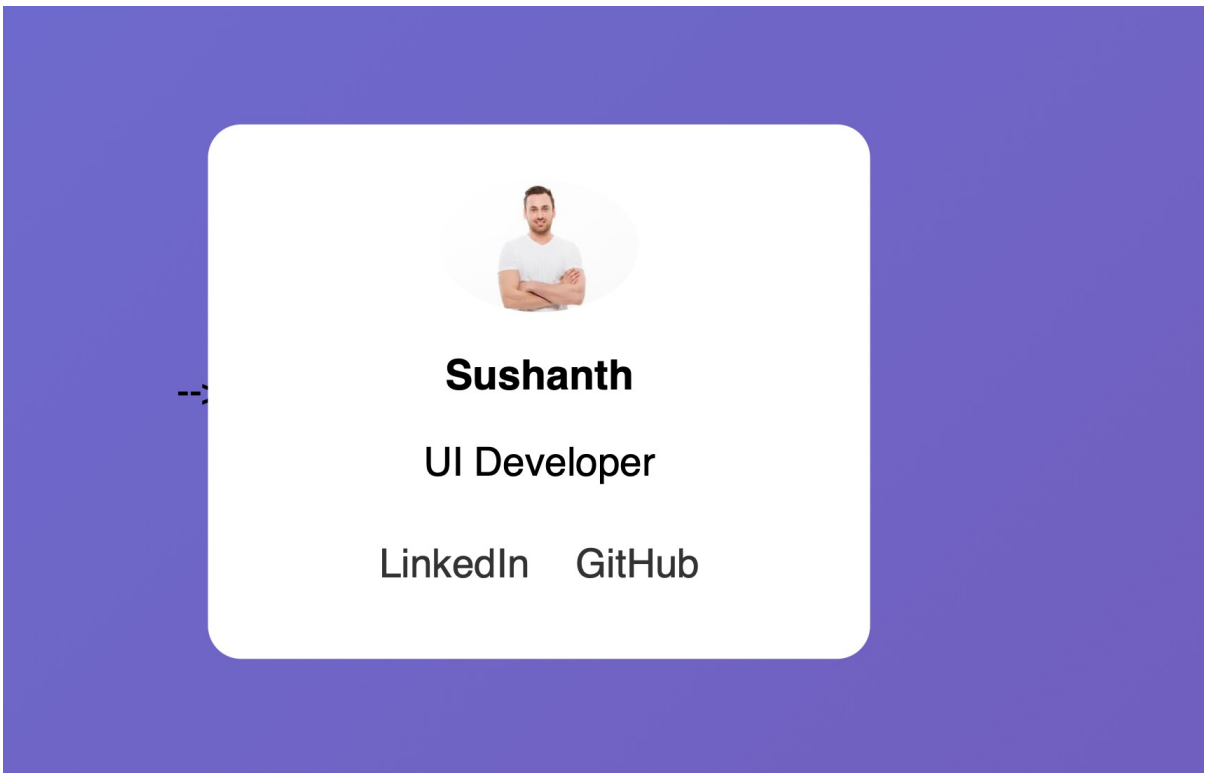
Requirements:

```html
<!-- Task 2 – Profile Card Generator
Create a profile card web page.
Requirements:
• Display user image, name, role, and social media links.
• Use AI to:
o Suggest modern UI colors and fonts.
o Design the card using Flexbox.
o Add hover animation and shadow effects  give the code -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Profile Card</title>
    <style>
        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            background-color: #f5f5f5;
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
            margin: 0;
        }
        .profile-card {
            background-color: #ffffff;
            border-radius: 10px;
            box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
            width: 300px;
            text-align: center;
            padding: 20px;
            transition: transform 0.3s ease, box-shadow 0.3s ease;
```

```html
<html lang="en">
<head>
    <style>
        }
        .profile-card:hover {
            transform: translateY(-10px);
            box-shadow: 0 8px 16px rgba(0, 0, 0, 0.2);
        }
        .profile-image {
            width: 100px;
            height: 100px;
            border-radius: 50%;
            object-fit: cover;
            margin-bottom: 15px;
        }
        .profile-name {
            font-size: 1.5em;
            color: #333333;
            margin-bottom: 5px;
        }
        .profile-role {
            font-size: 1em;
            color: #777777;
            margin-bottom: 15px;
        }
        .social-links a {
            text-decoration: none;
            color: #555555;
            margin: 0 10px;
            font-size: 1.2em;
        }
```

```html
<html lang="en">
<head>
    <style>
        .social-links a {
            font-size: 1.2em;
        }
        .social-links a:hover {
            color: #0073e6;
        }
    </style>
</head><body>
    <div class="profile-card">
        <img src="https://randomuser.me/api/portraits/men/75.jpg" alt="Profile Image" class="profile-image">
        <div class="profile-name">John Doe</div>
        <div class="profile-role">Web Developer</div>
        <div class="social-links">
            <a href="https://www.linkedin.com/in/johndoe" target="_blank">LinkedIn</a>
            <a href="https://twitter.com/johndoe" target="_blank">Twitter</a>
            <a href="https://github.com/johndoe" target="_blank">GitHub</a>
        </div>
    </div>
</body>
</html>
```

Output:



Explanation:

Designs a user profile card displaying image, name, role, and social links.
Uses Flexbox with hover effects and shadows for a modern UI look.

Task 3 -Develop a dynamic shopping cart system.

<!--
Develop a dynamic shopping cart system.
Requirements:

- Add, remove, and update product quantities.
- Display real-time total price.

Use AI to:

- Manage cart state using JavaScript objects.
- Persist cart data in localStorage.

Also include sample product images.

—>

Code:

lab(14.5)-3.html > html > body > script > updateCart

```html
<!-- Task 3 -Develop a dynamic shopping cart system.
Requirements:
• Add, remove, and update product quantities.
• Display real-time total price.
• Use AI to:
o Manage cart state using JavaScript objects.
o Persist cart data in localStorage. -->

<!-- i want images as well for the above prompt -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Dynamic Shopping Cart</title>
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 20px;
            display: flex;
            flex-direction: column;
            align-items: center;
            background-color: #f5f5f5;
        }

        h1 {
            margin-bottom: 30px;
        }

        .product {
            display: flex;
```

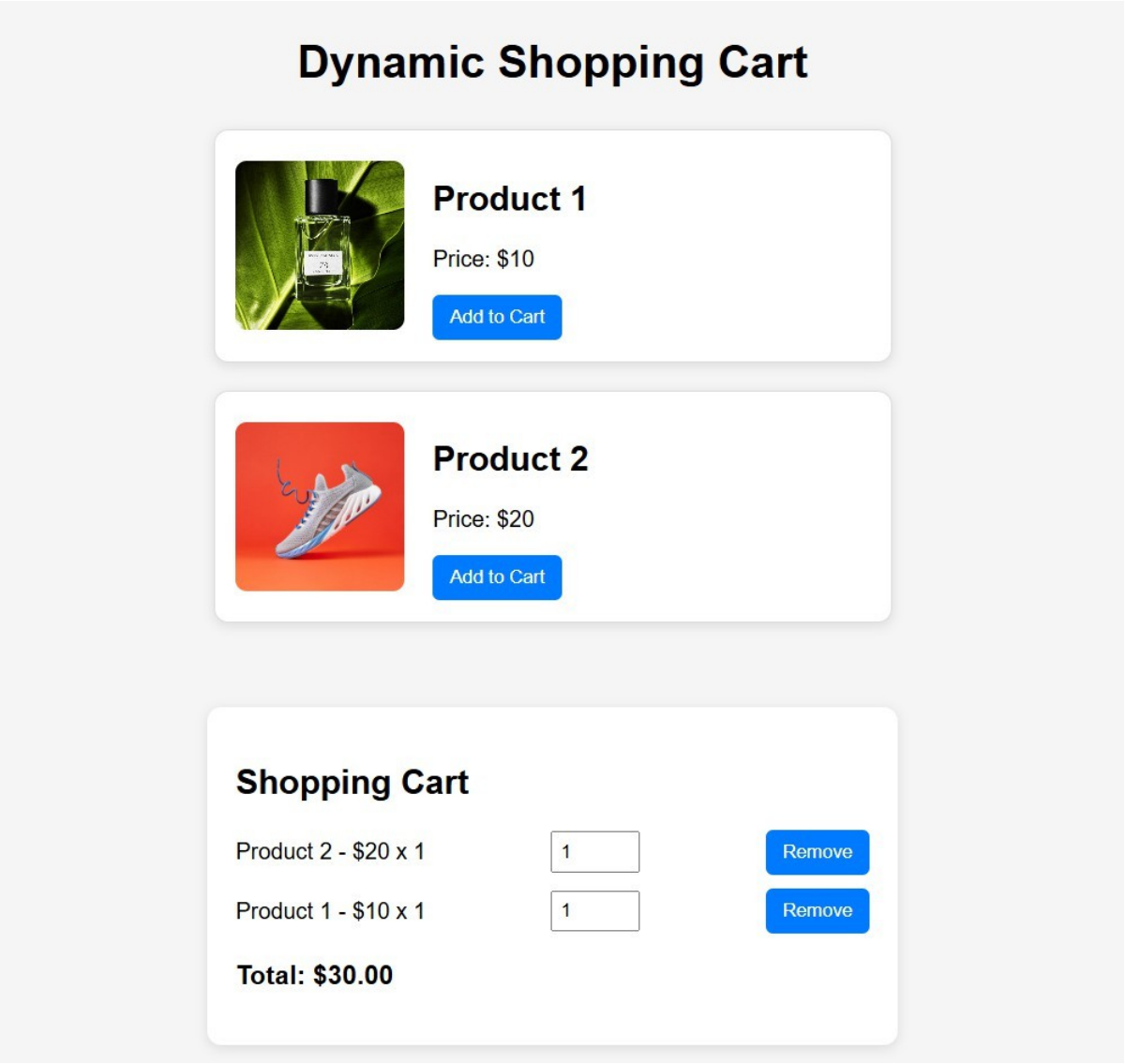
```html
    <style>
        }

        .product img {
            width: 120px;
            height: 120px;
            object-fit: cover;
            border-radius: 8px;
        }

        button {
            padding: 8px 12px;
            border: none;
            background-color: #007bff;
            color: white;
            border-radius: 5px;
            cursor: pointer;
        }

        button:hover {
            background-color: #0056b3;
        }

        .cart {
            margin-top: 40px;
            width: 450px;
            background: white;
            padding: 20px;
            border-radius: 10px;
            box-shadow: 0 2px 8px rgba(0,0,0,0.1);
```

```html
<body>

<h1>Dynamic Shopping Cart</h1>

<!-- Product 1 -->
<div class="product">
    <img src="https://i.pinimg.com/originals/35/94/79/3594793ee69d85ab8e82e780537fa83e.jpg" alt="Product 1">
    <div>
        <h2>Product 1</h2>
        <p>Price: $10</p>
        <button onclick="addToCart('Product 1', 10)">Add to Cart</button>
    </div>
</div>

<!-- Product 2 -->
<div class="product">
    <img src="https://d2v5dzhdg4zhx3.cloudfront.net/web-assets/images/storypages/primary/ProductShowcasesampleimages/JPEG/Product+Showcase
    <div>
        <h2>Product 2</h2>
        <p>Price: $20</p>
        <button onclick="addToCart('Product 2', 20)">Add to Cart</button>
    </div>
</div>

<!-- Cart Section -->
<div class="cart">
    <h2>Shopping Cart</h2>
    <div id="cart-items"></div>
    <h3>Total: $<span id="total-price">0</span></h3>
```

```html
<html lang="en">
<body>
<script>
```

```javascript
    function updateCart() {
        const cartItemsContainer = document.getElementById('cart-items');
        cartItemsContainer.innerHTML = '';
        let totalPrice = 0;

        for (const product in cart) {
            const item = cart[product];
            totalPrice += item.price * item.quantity;

            const cartItem = document.createElement('div');
            cartItem.classList.add('cart-item');

            cartItem.innerHTML = `
                <span>${product} - $${item.price} x ${item.quantity}</span>
                <input type="number" value="${item.quantity}" min="0"
                        onchange="updateQuantity('${product}', this.value)">
                <button onclick="removeFromCart('${product}')">Remove</button>
            `;

            cartItemsContainer.appendChild(cartItem);
        }

        document.getElementById('total-price').textContent = totalPrice.toFixed(2);
        localStorage.setItem('cart', JSON.stringify(cart));
    }

    updateCart();
</script>
```

Output:



Explanation:

Implements a dynamic cart system to add, remove, and update product quantities. Calculates total price in real-time and stores data using localStorage.

Task 4 – Role-Based Access Control Application

Create a role-based web application.

Requirements:

• Admin, Editor, User roles.

• Conditional UI rendering.

• Use AI to:

o Design access control logic.

o Maintain secure state transitions.

o Create scalable UI components.

Code:

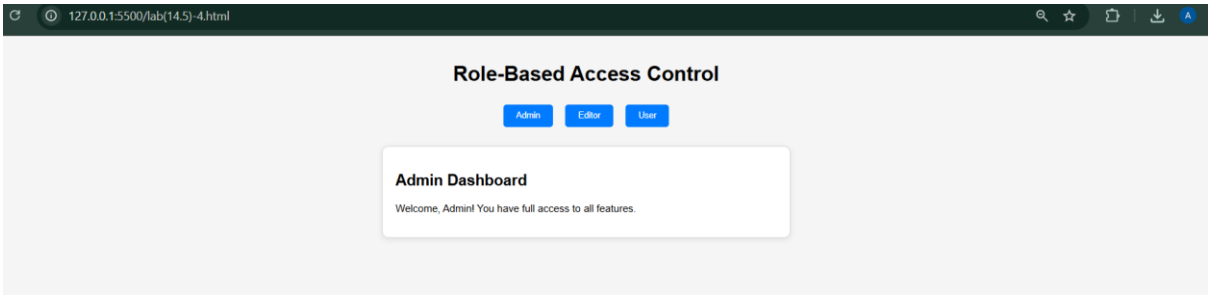
```
ab(14.5)-4.html > html > body > div.role-selection
<!-- Task 4 – Role-Based Access Control Application
Create a role-based web application.
Requirements:
• Admin, Editor, User roles.
• Conditional UI rendering.
• Use AI to:
o Design access control logic.
o Maintain secure state transitions.
o Create scalable UI components. -->
<!-- give the responsive webpage for the above task give code -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Role-Based Access Control</title>
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 20px;
            display: flex;
            flex-direction: column;
            align-items: center;
            background-color: #f5f5f5;
        }

        h1 {
            margin-bottom: 30px;
        }

        .role-selection {
```

```html
<html lang="en">
<head>
    <style>
        .role-selection {
            margin-bottom: 30px;
        }

        .role-selection button {
            padding: 10px 20px;
            border: none;
            background-color: #007bff;
            color: white;
            border-radius: 5px;
            cursor: pointer;
        }

        .role-selection button:hover {
            background-color: #0056b3;
        }

        .content {
            width: 100%;
            max-width: 600px;
            padding: 20px;
            border: 1px solid #ddd;
            border-radius: 10px;
            background-color: white;
            box-shadow: 0 2px 8px rgba(0,0,0,0.1);
        }

        @media (max-width: 600px) {
            .content {
```

lab(14.5)-4.html > html > body > div.role-selection

```html
<html lang="en">
<head>
    </style>
</head>
<body>
    <h1>Role-Based Access Control</h1>
    <div class="role-selection">
        <button onclick="setRole('Admin')">Admin</button>
        <button onclick="setRole('Editor')">Editor</button>
        <button onclick="setRole('User')">User</button>
    </div>
    <div class="content" id="content">
        Please select a role to see the content.
    </div>

    <script>
        function setRole(role) {
            const content = document.getElementById('content');
            if (role === 'Admin') {
                content.innerHTML = '<h2>Admin Dashboard</h2><p>Welcome, Admin! You have full access to all features.</p>';
            } else if (role === 'Editor') {
                content.innerHTML = '<h2>Editor Dashboard</h2><p>Welcome, Editor! You can edit content but have limited access to settin
            } else if (role === 'User') {
                content.innerHTML = '<h2>User Dashboard</h2><p>Welcome, User! You can view content but cannot make changes.</p>';
            }
        }
    </script>
</body>
</html>
```

Output:



Explanation:

Builds a system with Admin, Editor, and User roles.
Displays different UI content based on selected user role.

Task 5 – Multi-Language Website

Design a multilingual website.

Requirements:

• Switch languages dynamically.

• Persist language preference.

• Use AI to:

o Structure translation files.

o Handle RTL/LTR layouts.

o Improve accessibility.

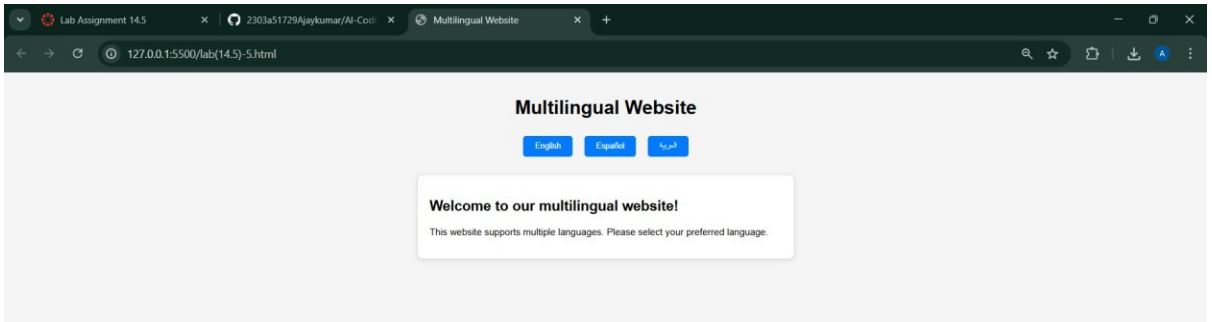
Code:

```html
<!-- Task 5 - Multi-Language Website
Design a multilingual website.
Requirements:
• Switch languages dynamically.
• Persist language preference.
• Use AI to:
o Structure translation files.
o Handle RTL/LTR layouts.
o Improve accessibility. -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Multilingual Website</title>
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 20px;
            display: flex;
            flex-direction: column;
            align-items: center;
            background-color: #f5f5f5;
        }

        h1 {
            margin-bottom: 30px;
        }

        .language-selection {
            display: flex;
```

```css
        .language-selection button {
            padding: 10px 20px;
            border: none;
            background-color: #007bff;
            color: white;
            border-radius: 5px;
            cursor: pointer;
        }

        .language-selection button:hover {
            background-color: #0056b3;
        }

        .content {
            width: 100%;
            max-width: 600px;
            padding: 20px;
            border: 1px solid #ddd;
            border-radius: 10px;
            background-color: white;
            box-shadow: 0 2px 8px rgba(0,0,0,0.1);
        }
    </style>
</head>
<body>
    <h1>Multilingual Website</h1>
    <div class="language-selection">
        <button onclick="setLanguage('en')">English</button>
```

```html
</head>
<body>
    <h1>Multilingual Website</h1>
    <div class="language-selection">
        <button onclick="setLanguage('en')">English</button>
        <button onclick="setLanguage('es')">Español</button>
        <button onclick="setLanguage('ar')">العربية</button>
    </div>
    <div class="content" id="content">
        <!-- Content will be dynamically updated based on language selection -->
    </div>

    <script>
        const translations = {
            en: {
                welcome: "Welcome to our multilingual website!",
                description: "This website supports multiple languages. Please select your preferred language."
            },
            es: {
                welcome: "¡Bienvenido a nuestro sitio web multilingüe!",
                description: "Este sitio web admite varios idiomas. Por favor, seleccione su idioma preferido."
            },
            ar: {
                welcome: "مرحبًا بكم في موقعنا متعدد اللغات!",
                description: "يدعم هذا الموقع عدة لغات. يرجى اختيار لغتك المفضلة."
            }
        };
```

Output:



Explanation:

Creates a website that allows users to switch between languages dynamically.
Stores user language preference and updates content accordingly.

Task 6 – Online Examination System

Create an online exam interface.

Requirements:

• Timer, questions, and submission.

• Prevent multiple submissions.

• Use AI to:

o Manage exam state.

o Improve exam UX.

o Ensure accessibility.

ode:

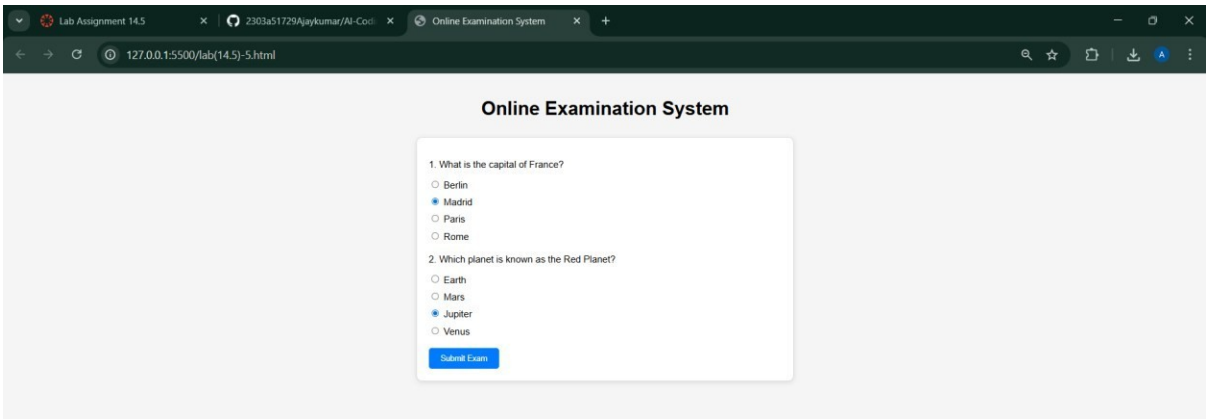
```html
<!-- Task 6 – Online Examination System
Create an online exam interface.
Requirements:
• Timer, questions, and submission.
• Prevent multiple submissions.
• Use AI to:
o Manage exam state.
o Improve exam UX.
o Ensure accessibility. -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Online Examination System</title>
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 20px;
            display: flex;
            flex-direction: column;
            align-items: center;
            background-color: #f5f5f5;
        }

        h1 {
            margin-bottom: 30px;
        }

        .exam-container {
            width: 100%;
```

```html
    <style>
        }

        .options {
            display: flex;
            flex-direction: column;
            gap: 10px;
        }

        button {
            padding: 10px 20px;
            border: none;
            background-color: #007bff;
            color: white;
            border-radius: 5px;
            cursor: pointer;
        }

        button:hover {
            background-color: #0056b3;
        }
    </style>
</head>
<body>
    <h1>Online Examination System</h1>
    <div class="exam-container">
        <div class="question">
            <p>1. What is the capital of France?</p>
            <div class="options">
                <label><input type="radio" name="q1" value="a"> Berlin</label>
```

```html
<html lang="en">
<head>
</head>
<body>
    <h1>Online Examination System</h1>
    <div class="exam-container">
        <div class="question">
            <p>1. What is the capital of France?</p>
            <div class="options">
                <label><input type="radio" name="q1" value="a"> Berlin</label>
                <label><input type="radio" name="q1" value="b"> Madrid</label>
                <label><input type="radio" name="q1" value="c"> Paris</label>
                <label><input type="radio" name="q1" value="d"> Rome</label>
            </div>
        </div>

        <div class="question">
            <p>2. Which planet is known as the Red Planet?</p>
            <div class="options">
                <label><input type="radio" name="q2" value="a"> Earth</label>
                <label><input type="radio" name="q2" value="b"> Mars</label>
                <label><input type="radio" name="q2" value="c"> Jupiter</label>
                <label><input type="radio" name="q2" value="d"> Venus</label>
            </div>
        </div>

        <button id="submit-btn">Submit Exam</button>
    </div>

    <script>
        const submitBtn = document.getElementById('submit-btn');
        let submitted = false;
```

```html
<body>

    <script>
        const submitBtn = document.getElementById('submit-btn');
        let submitted = false;

        submitBtn.addEventListener('click', () => {
            if (submitted) {
                alert('You have already submitted the exam.');
                return;
            }

            // Here you would typically gather the answers and send them to a server
            alert('Exam submitted successfully!');
            submitted = true;
        });
    </script>
</body>
</html>
```

Output:



Explanation:

Develops an online exam interface with questions and submission functionality.
Prevents multiple submissions and provides instant feedback.